

CMC Vellore | QIRAIL Laboratory

Graph Attention Networks in Plain English

A complete reference guide — how GATs work, how we use them for head and neck cancer outcome prediction, and answers to every question.

Project: RadGraph — Locoregional Recurrence Prediction in HNSCC

Reference: Bae et al., Radiology: Imaging Cancer, 2025

Part One: Building Blocks — What You Need to Know First

Before we can understand a Graph Attention Network, we need to understand three simpler ideas: what a graph is, what a neural network is, and what attention means. Each idea builds on the last. If you already know some of these, feel free to skip ahead.

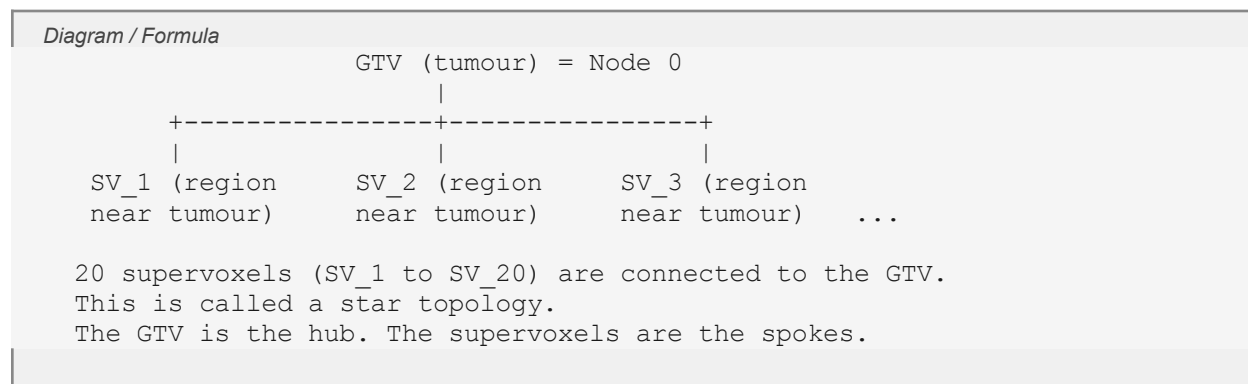
1. What is a Graph?

In mathematics, a graph is a set of objects (called nodes) connected by links (called edges). That is all it is. The word 'graph' here does not mean a bar chart or a pie chart — it means a network of connected things.

Examples of graphs you encounter every day:

- **Social network:** Each person is a node. A friendship is an edge. Facebook is a graph.
- **Road network:** Each city is a node. Each road is an edge. Google Maps uses a graph.
- **The internet:** Each website is a node. Each hyperlink is an edge.
- **Molecules:** Each atom is a node. Each chemical bond is an edge.

In our RadGraph project, the graph for each patient looks like this:



Each node has features — numbers that describe it. For our LR task, each node has 4 numbers describing its radiomic texture (things like how rough or smooth the tissue looks on CT).

Why use a graph instead of just a table?

A standard machine learning model treats each supervoxel independently. It sees a table with one row per supervoxel and 4 numbers per row. It has no idea that supervoxel 3 is 12mm away from supervoxel 7, or that both are adjacent to the tumour margin.

A graph-based model encodes the relationships between supervoxels explicitly. It can learn that the combination of certain features at certain positions around the tumour is predictive — not just individual features in isolation.

2. What is a Neural Network?

A neural network is a machine learning model loosely inspired by how the brain works. You feed it some numbers as input, it does a series of mathematical operations, and it produces an output — often a prediction.

The most important thing to know is that a neural network has parameters: numbers that it learns during training. These parameters are the 'weights' that control how much each input influences the output.

Simple example: predicting house prices

Input features: size (100 sqm), location (city centre), age (10 years)

Neural network does: $\text{output} = w_1 \cdot \text{size} + w_2 \cdot \text{location} + w_3 \cdot \text{age} + \text{bias}$

During training: the network adjusts w_1 , w_2 , w_3 to make predictions as accurate as possible.

After training: the network has learned that size matters most (w_1 is large) and age matters least (w_3 is small).

A neural network learns by comparing its predictions to the true answers and adjusting its weights to reduce the error. This process is called backpropagation and gradient descent.

What is a layer in a neural network?

A layer is one set of these weight-multiplication-and-sum operations. Most neural networks have multiple layers stacked on top of each other. Each layer transforms the data a little further. Early layers learn simple patterns (edges in an image, for example). Later layers combine those simple patterns into complex ones (a face, for example).

3. What is Attention?

Attention is a mechanism that lets a model focus on the most relevant parts of its input rather than treating everything equally.

Everyday analogy

Imagine you are at a noisy party. Ten people are talking at once.

You do not listen to all ten equally. You pay most attention to the person talking to you, a little attention to the person next to them, and almost no attention to people far away.

That selective listening is attention. You assign a weight to each person.

High weight = listen carefully. Low weight = mostly ignore.

In a neural network, attention works the same way. When updating one node's information, the model computes a score for every other connected node: how relevant is this neighbour to me right now? Those scores become attention weights, and the model aggregates information from neighbours proportionally to those weights.

The key innovation of attention over older methods is that these weights are not fixed. They are computed dynamically from the data itself, and they can be different for every input example.

Part Two: Graph Attention Networks — How They Work

Now we have all three ingredients: graphs, neural networks, and attention. A Graph Attention Network (GAT) combines all three. It is a neural network that operates on graphs and uses attention to decide how much each neighbour matters.

4. Message Passing — The Core Idea

The fundamental operation in any graph neural network is called message passing. It works like this:

1. Each node looks at its neighbours.
2. Each node collects (or 'receives messages from') the features of those neighbours.
3. Each node updates its own features based on what it collected.
4. Repeat for several rounds (layers).

After enough rounds of message passing, each node's feature vector contains information not just about itself, but about its neighbourhood — and through the neighbourhood, about the broader graph.

Physical analogy — rumour spreading in a village

Imagine a village of 21 houses. The village square (GTV) is connected to 20 surrounding houses (supervoxels).

Day 1 (Layer 1): Everyone shouts their news to the square. The square now knows everyone's news.

Day 2 (Layer 2): The square shares a summary back to each house. Now every house knows what the square knows.

Day 3 (Layer 3): Everyone shouts to the square again — but now their news includes what they heard from the square.

After 5 days (5 layers), each house has indirectly heard about every other house, even though houses only talk directly to the square and never to each other.

5. How GAT is Different from a Basic GNN

A basic Graph Neural Network (GNN) collects information from neighbours by simply averaging them. If a node has 3 neighbours, it adds up all their feature vectors and divides by 3.

The problem with this: it treats all neighbours as equally important. A supervoxel right at the tumour invasion front is treated the same as one deep in muscle tissue.

GAT fixes this with attention weights. Instead of:

Diagram / Formula

$$\text{new_GTV} = (\text{SV_1} + \text{SV_2} + \dots + \text{SV_20}) / 20 \quad (\text{basic GNN})$$

GAT computes:

Diagram / Formula

$$\text{new_GTV} = \alpha_1 \cdot \text{SV}_1 + \alpha_2 \cdot \text{SV}_2 + \dots + \alpha_{20} \cdot \text{SV}_{20}$$

Where α_1 through α_{20} are attention weights that sum to 1. These weights are learned during training and can be different for every patient.

6. How Attention Weights Are Computed — Step by Step

Here is the complete calculation, with a concrete example at every step. We will work through how the GTV computes the attention weight for supervoxel SV_3 .

Step 1 — Feature projection

The raw feature vectors are too small and specific to compare directly. Both the GTV and SV_3 first have their features projected into a higher-dimensional space through a learned weight matrix W :

Diagram / Formula

```
GTV has features:  [f1, f2, f3, f4]  (4 numbers)
W is a matrix:      shape (4 x 16)
Projected GTV:      W * [f1,f2,f3,f4] = [16 numbers]
Projected SV_3:     W * [f1,f2,f3,f4] = [16 numbers]
```

Think of W as a translation — converting raw radiomic measurements into a common 'language' that the attention mechanism understands.

Step 2 — Concatenation

The projected GTV vector and the projected SV_3 vector are joined end-to-end into one long vector:

Diagram / Formula

```
[GTV projected (16 numbers) || SV_3 projected (16 numbers)]
Result: 32 numbers total
```

The symbol `||` means concatenation (placing end to end, not addition).

Why concatenate? Because we want the attention score to depend on BOTH the GTV and the supervoxel. A supervoxel that is very different from the GTV might deserve more or less attention depending on the context.

Step 3 — Scoring

A small learned vector called 'a' (just 32 numbers) is multiplied element-by-element with the concatenated vector and the results are summed to produce one raw score number:

Diagram / Formula

```
raw_score = a dot [projected_GTV || projected_SV_3]
            = a[1]*val[1] + a[2]*val[2] + ... + a[32]*val[32]
            = one single number
```

This number can be anything — positive, negative, large, or small. It is the raw measure of how relevant SV_3 is to the GTV.

Step 4 — LeakyReLU

The raw score passes through a LeakyReLU function. This just means: if the score is positive, keep it. If it is negative, multiply it by a small number (0.2 in our case) rather than setting it to zero. This prevents the network from completely ignoring certain supervoxels during early training.

Diagram / Formula

```
LeakyReLU(x) :
  if x >= 0:   output = x
  if x < 0:    output = 0.2 * x
```

Example:

```
raw_score = 1.4  ->  output = 1.4
raw_score = -0.8 ->  output = -0.16  (not -0.8, not 0)
```

Step 5 — Softmax normalisation

The same calculation is done for all 20 supervoxels, giving 20 raw scores. These are then passed through softmax to convert them into weights that sum to 1.0:

Diagram / Formula

```
Scores before softmax: [1.4, 0.8, -0.2, 1.1, 0.3, ...]  (20 numbers)

Softmax(score_k) = exp(score_k) / sum( exp(score_j) for j = 1 to 20 )

Weights after softmax: [0.18, 0.11, 0.04, 0.14, 0.07, ...]
                      sum = 1.00  (always, by design)
```

Supervoxels with higher raw scores get higher weights. Supervoxels with lower scores get lower weights. But all weights are positive and sum to 1.

Step 6 — Weighted aggregation

Finally, the GTV's new feature vector is the sum of all supervoxel feature vectors, each multiplied by its attention weight:

Diagram / Formula

```

new_GTV_features = alpha_1 * SV_1_features
                  + alpha_2 * SV_2_features
                  + ...
                  + alpha_20 * SV_20_features

```

This is one iteration of one GAT layer for the GTV node.

7. Multiple Attention Heads

Our model uses 4 attention heads. This means the entire attention calculation above — the weight matrix W , the attention vector a , and the resulting weights — runs 4 times in parallel, each time with different learned parameters.

Each head can specialise in a different aspect of relevance:

- Head 1 might focus on supervoxels with similar texture to the GTV
- Head 2 might focus on supervoxels at specific spatial locations
- Head 3 might focus on supervoxels with extreme feature values
- Head 4 might focus on some combination the model invented during training

The outputs of all 4 heads are concatenated. So if each head produces 16 numbers, concatenating 4 heads gives 64 numbers. In the final layer, the heads are averaged rather than concatenated (since we want a fixed-size output for the readout step).

8. The Five Layers

Our model for the LR task has 5 GAT layers stacked on top of each other. Each layer is one complete pass of: compute attention weights, aggregate from neighbours, update node features.

Why 5? More layers means each node gathers information from a wider neighbourhood. With our star graph:

Layer	What the GTV knows after this layer	What each SV knows after this layer
0 (input)	Only its own 4 features	Only its own 4 features
1	Weighted mix of all 20 SV features	What the GTV looked like at Layer 0
2	Updated SVs that heard from Layer 1 GTV	GTV's Layer 1 summary (which included all SVs)
3	SVs that now carry Layer 2 GTV information	GTV's Layer 2 summary (indirect SV-SV info)
4	Deep mixture of cross-SV information	Rich context from the whole graph

Layer	What the GTV knows after this layer	What each SV knows after this layer
5	Full 5-round distillation of the graph	Every SV has indirectly influenced every other

More layers does not always mean better. After too many layers, all node vectors start looking the same — a problem called over-smoothing. The paper found through experiments that 5 layers is the sweet spot for the LR task (Table S2).

Part Three: How We Use GAT in RadGraph

Now that we understand what a GAT is, let us walk through exactly how we apply it to predict locoregional recurrence (LR) in head and neck cancer patients at CMC Vellore.

9. The Clinical Problem

Head and neck squamous cell carcinoma (HNSCC) is treated with radiation therapy. After treatment, some patients develop locoregional recurrence — the cancer comes back in the same region — within 2 years. Identifying which patients are at high risk before treatment would allow doctors to intensify therapy for those patients.

Our goal: given a patient's pretreatment CT scan, predict whether they will have locoregional recurrence within 2 years.

Our dataset: 120 HNSCC patients from CMC Vellore with CT scans, RT structure files (GTV delineations), clinical variables, and 2-year follow-up outcomes.

10. The Complete Pipeline — End to End

The pipeline has 5 stages. Each stage's output feeds into the next.

Stage	What it does	Input	Output
1 Preprocessing	Resample CT to 1mm spacing, define 5cm peritumoral region	DICOM CT + RT struct	ct_array, gtv_array, region_array
2 Supervoxels	Segment peritumoral region into ~100 sub-regions (supervoxels)	ct_array + region_array	supervoxel label map
3 Feature extraction	Extract 83 radiomic texture features from GTV and each supervoxel	CT + supervoxel labels	83 features per supervoxel, per patient
4 Graph building	Construct a star graph per patient (GTV hub + top 20 SVs)	Feature vectors + centroids	PyG Data object per patient
5 GAT training	Train the 5-layer GAT to predict recurrence	Patient graphs + clinical data	Trained model, AUC, attention maps

11. Why Supervoxels? Why Not Just Use the GTV?

Traditional radiomics extracts features from the whole GTV as one single region. This produces one feature vector per patient. It tells you about the tumour but nothing about the tissue around it.

The RadGraph paper argues that the microenvironment surrounding the tumour — the peritumoral tissue — carries independent prognostic information. Regions of the peritumoral tissue that look aggressive on CT may indicate areas where tumour cells are spreading, even if they are not yet detectable as tumour.

Supervoxels are small, locally homogeneous sub-volumes of that peritumoral region. Each one gets its own radiomic feature vector. This allows the model to ask: which regions around the tumour are most informative for predicting recurrence?

12. Why a Star Graph?

We could connect supervoxels to each other (and Joseph Bae's original code does this). But the paper describes a star graph: GTV at the centre connected to each supervoxel, with no direct supervoxel-to-supervoxel edges.

The reasoning is:

- The GTV is the clinical anchor. The model's prediction should be GTV-centric.
- The peritumoral supervoxels provide context for the GTV. Their relevance should be judged relative to the GTV.
- With 5 layers of message passing, information still flows indirectly between supervoxels via the GTV hub.

13. How the Top-20 Supervoxels Are Selected

We generate ~100 supervoxels per patient, but only use the 20 most similar to the GTV in the graph. Similarity is measured by cosine similarity of radiomic features.

What is cosine similarity?

Cosine similarity measures the angle between two feature vectors.

If both vectors point in the same direction (angle = 0 degrees), similarity = 1.0 (identical).

If they are perpendicular (angle = 90 degrees), similarity = 0.0 (unrelated).

We pick the 20 supervoxels whose feature vectors point most in the same direction as the GTV vector.

These are the supervoxels that 'look most like' the tumour in terms of texture.

The hypothesis is that these peritumoral regions may represent areas of microinvasion.

14. Feature Normalisation Before Graph Construction

Raw radiomic feature values have very different scales. One feature might range from 0 to 1000; another from 0 to 0.001. Mixing these directly would make the model dominated by features with large numerical values.

We normalise features using a StandardScaler fitted on training patients only. This converts every feature to have a mean of 0 and a standard deviation of 1 (z-score normalisation). Values beyond ± 5 standard deviations are clipped.

Critically, the scaler is fitted on training patients only and then applied to validation and test patients. This prevents information from the test set from leaking into the normalisation.

15. Clinical Feature Fusion

The model also uses 8 clinical variables: age, sex, HPV status, AJCC stage, ECOG status, concurrent chemoradiation, tumour subsite, and tumour volume.

These are not fed into the graph. Instead, after the GTV readout produces a 16-dimensional vector, the clinical features are concatenated to it before the final classifier:

Diagram / Formula

```

GTV readout vector  (16 numbers)
Clinical features    ( 8 numbers)
-----
Concatenated:       (24 numbers)  -> Linear  -> 1 number  -> sigmoid
-> P(LR)

```

16. Training the Model

Training means finding the weight matrices W and attention vectors a (and all other parameters) that make the model's predictions as accurate as possible on the training data.

This is done by:

5. Feeding all training patients through the model
6. Computing how wrong the predictions are (using a loss function: Binary Cross-Entropy)
7. Computing how each parameter contributed to the error (backpropagation)
8. Adjusting each parameter slightly to reduce the error (gradient descent with Adam optimiser)
9. Repeating for 100 epochs (100 complete passes through the training data)

Handling class imbalance

Our dataset has 34 patients with recurrence and 87 without — roughly 28% positive. If we trained naively, the model might learn to always predict 'no recurrence' and achieve 72% accuracy while being completely useless clinically.

We use an intermediate sampling strategy: we partially oversample the minority class (recurrence) and partially undersample the majority class (no recurrence) so the training set is more balanced. The validation and test sets are kept at their original distributions.

Early stopping

We stop training when the validation AUC has not improved for 15 consecutive epochs. The model weights from the best epoch are saved and used for evaluation. This prevents overfitting — memorising the training data rather than learning generalisable patterns.

Part Four: The Three Original Questions

Q1. What is the final readout layer in the GAT?

After 5 layers of message passing, every node has a feature vector. The model needs to produce one prediction per patient. This requires collapsing 21 node vectors into 1.

Most graph neural networks do this by averaging all node vectors. RadGraph does something more targeted: it reads out only the GTV node (node index 0) and ignores the 20 supervoxel vectors.

Why only the GTV?

By the time the GAT finishes 5 layers, the GTV node has gathered and filtered information from all 20 supervoxels through 5 rounds of attention-weighted message passing.

The GTV's final feature vector is a compact summary of the entire tumour microenvironment, filtered through the network's own learned sense of relevance.

Reading out only the GTV is not throwing away information.
It is using the single most information-rich node in the graph.

Diagram / Formula

```

21 nodes enter the final GAT layer
|
Readout: extract features of node index 0 only (the GTV)
|
1 vector of 16 numbers
|
Concatenate: 8 clinical features -> 24 numbers
|
Linear layer: 24 -> 1 (one logit)
|
Sigmoid: convert logit to probability 0.0 to 1.0
|
P(locoregional recurrence within 2 years)

```

Q2. How does the attention mechanism assign weights to nodes?

At each GAT layer, the model computes an attention weight for every edge in the graph. In our star graph, there are 20 edges from the GTV to supervoxels. Each gets a weight between 0 and 1.

The calculation has 6 steps, all explained in detail in Part Two (Section 6). In brief:

10. Project both nodes' features through a shared weight matrix W
11. Concatenate the two projected vectors
12. Dot-product with a learned attention vector a to get one raw score number
13. Apply LeakyReLU

14. Apply softmax across all 20 supervoxels to get weights that sum to 1.0
15. Aggregate supervoxel features weighted by these attention weights

These attention weights are not fixed. They are computed fresh for every patient based on that patient's own radiomic features. This means two patients with very different tumour characteristics will have very different attention maps — even though they share the same trained model.

Concrete example

After training, for patient HN1004:

SV_3 (invasive margin region): attention weight = 0.18
 SV_7 (heterogeneous adjacent region): attention weight = 0.15
 SV_12 (deep homogeneous muscle): attention weight = 0.03
 SV_19 (bone interface): attention weight = 0.02

The model has learned to focus on the invasive margin and ignore deep muscle and bone. These weights become the spatial attention map for this patient.

Q3. How do supervoxels exchange information during the GAT process?

This question reveals the core power of multi-layer GATs. In our star topology, supervoxels are NOT directly connected to each other. Yet after 5 layers, each supervoxel's feature vector has been influenced by all other supervoxels in the graph.

How? Through the GTV hub, across multiple layers.

Diagram / Formula

LAYER 0 (before any processing)
 GTV knows: only its own 4 features
 SV_3 knows: only its own 4 features
 SV_7 knows: only its own 4 features

LAYER 1

GTV sends its features to all SVs, receives from all SVs
 GTV now knows: weighted combination of all 20 SV features
 SV_3 now knows: its own features + GTV original features

LAYER 2

GTV sends its Layer 1 summary (which contains all SV signals) back to all SVs
 SV_3 now knows: its features + GTV Layer 1 summary (which includes SV_7 signal)
 SV_3 has now INDIRECTLY heard from SV_7, via the GTV!

LAYERS 3, 4, 5

This indirect communication continues, getting richer with each layer.
 By Layer 5: every node has been influenced by every other node.

This is the fundamental reason we use a graph neural network rather than extracting features independently from each supervoxel. The model can learn patterns that depend on the spatial configuration of multiple supervoxels around the GTV — not just the features of each supervoxel in isolation.

Part Five: Frequently Asked Questions

This section addresses questions that commonly arise when explaining this work to clinicians, students, and collaborators who are new to machine learning.

Q4. How is this different from regular radiomics?

Regular radiomics extracts one feature vector from the whole tumour (GTV) and trains a Random Forest or logistic regression on those features. The RadGraph approach is different in three ways:

Aspect	Traditional Radiomics	RadGraph GAT
Region of interest	Whole GTV only	GTV + 20 peritumoral supervoxels
Feature count	~93 features, 1 per patient	~4 features x 21 nodes per patient
Spatial information	None — one aggregate vector	Explicit — encoded in graph edges
Model	Random Forest, SVM, or logistic regression	Graph Attention Network
Interaction between regions	Not captured	Learned through message passing

Q5. What is the AUC and why do we use it?

AUC stands for Area Under the Receiver Operating Characteristic Curve. It measures how well the model separates the two classes (recurrence vs no recurrence).

An AUC of 0.5 means the model is no better than random guessing — like flipping a coin. An AUC of 1.0 means the model is perfect. An AUC of 0.75 means that if you pick one random patient who had recurrence and one who did not, the model ranks the recurrence patient as higher risk 75% of the time.

We use AUC rather than accuracy because our dataset is imbalanced (28% positive). A model that always predicts 'no recurrence' would have 72% accuracy, but an AUC of only 0.5 — immediately revealing it is useless.

Expected performance for this project

Clinical baseline (stage, HPV, etc. only): AUC 0.60 - 0.65

Traditional radiomics baseline (whole GTV): AUC 0.65 - 0.75

RadGraph GAT (spatial graph): AUC 0.75 - 0.83

The paper reports AUC 0.83 on their external test set.

We expect slightly lower on our CMC Vellore cohort due to smaller sample size (91 patients vs 500+).

Q6. What are the attention maps and how do we interpret them?

After training, we extract the attention weights from the final GAT layer for every patient in the test set. These weights tell us which supervoxels the model focused on most when making each patient's prediction.

Because we know the 3D spatial location of every supervoxel (its centroid coordinates), we can map the attention weights back onto the CT scan. This creates a spatial attention map: a heatmap overlaid on the CT showing which regions around the tumour the model found most predictive.

Clinical interpretation: regions with high attention weight that are associated with high predicted recurrence risk may correspond to areas of:

- Peritumoral invasion not visible on CT
- High-risk tissue microenvironment
- Early spread into lymph node pathways

These maps are potentially useful for radiotherapy planning — they could guide dose escalation to high-risk regions.

Q7. Why do we use 4 features for the LR task instead of all 93?

The paper (Table S3) identified 4 specific radiomic features from the GLCM and GLSZM texture families as most predictive of locoregional recurrence, selected using mRMR (Minimum Redundancy Maximum Relevance) feature selection.

Using fewer, better-selected features has several advantages:

- Reduces overfitting — the model has fewer numbers to memorise
- Speeds up training and inference
- Makes the model more interpretable — we know which specific texture properties matter
- With only 91 patients, using 93 features would severely overfit

The 4 LR features are: GLSZM Gray Level Non-Uniformity, GLCM Inverse Difference, GLCM Inverse Difference Normalised, and GLSZM Zone Entropy. These all relate to how uniform or heterogeneous the tissue texture appears on CT.

Q8. What does the model actually learn during training?

The model learns three types of parameters:

Parameter type	What it is	What it controls
W matrices (one per layer per head)	16x4 weight matrices	How to project raw features into the attention space
Attention vectors a (one per layer per head)	32-number vectors	How to score the relevance of each supervoxel to the GTV
Classifier weights (final linear layer)	24x1 weight vector	How to combine GTV readout and clinical features into a risk score

The total number of trainable parameters in our model is approximately 5,000 to 8,000 numbers — quite small by modern deep learning standards. This is intentional: with only 91 patients, we need a small, well-regularised model to avoid overfitting.

Q9. What happens if a patient has fewer than 20 supervoxels?

The graph construction uses $K = \min(20, n_{\text{supervoxels}})$. If a patient has only 15 valid supervoxels (for example, a very small peritumoral region), the graph will have 16 nodes (1 GTV + 15 supervoxels) and 30 edges instead of 40.

This is handled gracefully — PyTorch Geometric's DataLoader handles variable-size graphs in the same batch. The model is not confused by different numbers of nodes; it processes whatever the graph contains.

Q10. What is over-smoothing, and how do we avoid it?

Over-smoothing is a problem that occurs when you stack too many GAT layers. After many layers of message passing, every node's feature vector becomes a weighted average of all other nodes. They all start to look the same. The individual signatures of each supervoxel are washed out.

We avoid this with:

- Dropout (0.67 probability) between layers — randomly zeros out some features, preventing over-reliance on any single signal
- LayerNorm — normalises each node's features within each graph, preventing scale issues
- Only 5 layers — not more than needed to cover the graph diameter

Q11. How is the model evaluated fairly?

We use a strict train/validation/test split:

- 70% of patients (about 85) go into the training set — the model sees their labels during training
- 15% of patients (about 14) go into the validation set — used to select the best model checkpoint and tune hyperparameters
- 15% of patients (about 14) go into the test set — the model never sees these during any part of training or selection

The test AUC is the final reported performance. We also compute a 95% bootstrap confidence interval (1000 resamples) to quantify uncertainty — important with such a small test set.

The split is stratified: the 28% recurrence rate is preserved in all three splits, so the model is not accidentally trained on a subset with very different class balance.

Q12. What are the limitations of this approach?

Being honest about limitations is important, especially in a clinical setting:

- **Small dataset:** 91 patients is small for a deep learning model. Results may not generalise to other hospitals or patient populations. This is why external validation on a larger cohort (such as RADCURE) is important.

- **CT resolution:** Our CTs have 3mm slice thickness, resampled to 1mm isotropic. Some fine-grained texture features may be affected by interpolation artefacts.
- **Single institution:** All patients are from CMC Vellore. Different scanner protocols, treatment practices, and patient demographics at other institutions may affect model performance.
- **2-year endpoint only:** We define recurrence as occurring within 2 years. Patients censored before 2 years with no event are excluded. This creates selection bias toward patients who were followed for a long time.
- **Black box:** Despite the attention maps, the model's decisions are not fully explainable. A high attention weight on a supervoxel does not necessarily mean that region caused recurrence.

Part Six: Quick Reference

Model Architecture Summary

Component	Configuration (LR task)	Why
Input features per node	4 (Table S3)	GLCM and GLSZM texture features
GAT layers	5	Optimal depth found by hyperparameter search
Hidden dimension	16 per head	Small to avoid overfitting on 91 patients
Attention heads	4	Multiple notions of relevance
Dropout	0.67	Heavy regularisation for small dataset
Learning rate	0.0004	Table S2 value
Batch size	64	Table S2 value
Sampling strategy	Intermediate	Balances 28% positive class
Early stopping patience	15 epochs	Prevents overfitting
Readout	GTV node only (index 0)	GTV as information hub
Normalisation	LayerNorm per graph	Better than BatchNorm for variable-size graphs
Loss function	BCEWithLogitsLoss	Binary classification with imbalance handling
Clinical features	8 (concatenated after readout)	HPV, stage, age, etc.

Terminology Glossary

Term	Plain English definition
Graph	A set of objects (nodes) connected by links (edges)
Node	One object in the graph. In our case: GTV or one supervoxel.
Edge	A connection between two nodes. In our case: GTV is connected to each supervoxel.
Feature vector	A list of numbers describing a node. In our case: 4 radiomic measurements.
Message passing	The process of nodes sharing information with their neighbours
Attention weight	A number between 0 and 1 saying how much to weight a neighbour's information

Term	Plain English definition
Softmax	A function that converts any set of numbers into positive weights that sum to 1
Readout / pooling	The step where many node vectors are collapsed into one graph-level vector
Epoch	One complete pass through the entire training dataset
AUC	A metric (0 to 1) measuring how well the model separates two classes
Overfitting	When a model memorises the training data instead of learning general patterns
Dropout	Randomly zeroing out some features during training to prevent overfitting
LayerNorm	Normalising each node's features to have mean 0 and variance 1, per graph
Backpropagation	The algorithm for computing how each parameter contributed to the model's error
Gradient descent	The algorithm for adjusting parameters to reduce the model's error
Cosine similarity	A measure of how similar two vectors are in terms of their direction (0 to 1)
mRMR	A feature selection method: picks features that are relevant but not redundant with each other
Supervoxel	A small, locally homogeneous 3D region of the CT scan, used as a graph node
Star topology	A graph where one central node (hub) is connected to all others (spokes)
GTV	Gross Tumour Volume — the tumour region delineated by the radiation oncologist on CT

Key Equations

Diagram / Formula

ATTENTION WEIGHT (for supervoxel k connected to GTV):

$$\alpha_k = \text{softmax}(\text{LeakyReLU}(a \cdot [W \cdot h_{\text{GTV}} \parallel W \cdot h_{\text{SV}_k}]))$$

where:

W = learned weight matrix (projects features into attention space)

a = learned attention vector

h = feature vector of a node

|| = concatenation

GTV UPDATE (weighted aggregation):

```
new_h_GTV = ELU( sum( alpha_k * W * h_SV_k ) for k = 1..20 )
```

```
FINAL PREDICTION:
```

```
-----
```

```
logit = W_cls * [h_GTV_final || clinical_features] + bias
```

```
P(LR) = sigmoid(logit) = 1 / (1 + exp(-logit))
```

Appendix: Future Questions

This section will be updated each time a new question is asked. Each new question will be added in the same format as Part Four and Part Five above: plain English explanation, example, and key takeaway.

Q13: *(to be added)*

Q14: *(to be added)*